

Member visit and apply

Document #: P2637R0
Date: 2022-09-05
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[1.1 `std::visit`](#)

[1.2 `std::apply`](#)

[1.3 `std::visit\_format\_arg`](#)

[1.4 Implementation](#)

[2 Wording](#)

[3 References](#)

§ 1 Introduction

The standard library currently has three free function templates: `std::visit`, `std::apply`, and `std::visit_format_arg`. The goal of this paper is to add member function versions of each of them, simply for ergonomic reasons. This paper adds no new functionality that did not exist before.

§ 1.1 `std::visit`

`std::visit` is a variadic function template, which is the correct design since binary (and more) visitation is a useful and important piece of functionality. However, the common case is simply unary visitation. Even in that case, however, a non-member function was a superior implementation choice for forwarding const-ness and value category.^{[1](#)}

But this decision logic changes in C++23 with the introduction of deducing `this` [[P0847R7](#)]. Now, it is possible to implement unary `visit` as a member function without any loss of functionality. We simply gain better syntax:

Existing	Proposed
<pre>std::visit(overload{ [](int i){ std::print("i={}\n", i); }, [](std::string s){ std::print("s= {?:}\n", s); } }, value);</pre>	<pre>value.visit(overload{ [](int i){ std::print("i={}\n", i); }, [](std::string s){ std::print("s= {?:}\n", s); } });</pre>

§ 1.2 std::apply

`std::apply`, also added in C++17, is also a non-member function template. It takes a single *tuple-like* object and a callable, and its interface otherwise mirrors `std::variant`. I am not sure why it takes the function first and the tuple second, even though the tuple is the subject of the operation.

`std::apply` originally needed to be a non-member function for one of the same reasons as `std::visit`: proper forwarding of const-ness and value category. But, as with `std::visit`, this can now easily be made a member function template. It's just that we have to add it to multiple types: `pair`, `tuple`, `array`, and `subrange`.

Existing	Proposed
<pre>int sum = std::apply([](auto... args){ return (0 + ... + args); }, elements);</pre>	<pre>int sum = elements.apply([](auto... args){ return (0 + ... + args); });</pre>

§ 1.3 std::visit_format_arg

One of the components of the format library is `basic_format_arg<Context>` (see 22.14.7.1 [\[format.arg\]](#)), which is basically a `std::variant`. As such, it also needs to be visited in order to be used. To that end, the library provides:

```
template<class Visitor, class Context>
decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context>
    arg);
```

But here, the only reason `std::visit_format_arg` is a non-member function was to mirror the interface for `std::visit`. There is neither multiple visitation nor forwarding of value category or const-ness here. It could always have been a member function without any loss of functionality. With deducing `this`, it can even be by-value member function.

This example is from the standard itself:

Existing	Proposed
<pre> auto format(S s, format_context& ctx) { int width = visit_format_arg([](auto value) -> int { if constexpr (!is_integral_v<decltype(value)>) throw format_error("width is not integral"); else if (value < 0 value > numeric_limits<int>::max()) throw format_error("invalid width"); else return value; }, ctx.arg(width_arg_id)); return format_to(ctx.out(), " {0:x<{1}}", s.value, width); } </pre>	<pre> auto format(S s, format_context& ctx) { int width = ctx.arg(width_arg_id).visit([] (auto value) -> int { if constexpr (!is_integral_v<decltype(value)>) throw format_error("width is not integral"); else if (value < 0 value > numeric_limits<int>::max()) throw format_error("invalid width"); else return value; }); return format_to(ctx.out(), " {0:x<{1}}", s.value, width); } </pre>

The proposed name here is just `visit` (rather than `visit_format_arg`), since as a member function we don't need the longer name for differentiation.

§ 1.4 Implementation

In each case, the implementation is simple: simply redirect to the corresponding non-member function. Member `visit`, for instance:

```

template <class... Types>
class variant {
public:
    template <class Self, class Visitor>
        requires convertible_to<add_pointer_t<Self>, variant const*>
        constexpr auto visit(this Self&& self, Visitor&& vis) -> decltype(auto) {
            return std::visit(std::forward<Visitor>(vis), std::forward<Self>(self));
        }
};

```

The constraint is to reject those cases where a type might inherit privately inherit from `variant`. Those cases aren't supported by `std::variant` either.

§ 2 Wording

Add to 22.3.2 [\[pairs.pair\]](#):

```

namespace std {
    template<class T1, class T2>
    struct pair {
        // ...

        constexpr void swap(pair& p) noexcept(see below);
    };
}

```

```

    constexpr void swap(const pair& p) const noexcept(see below);
+   template<class Self, class F>
+   constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
        below);
    };
}

```

And to 22.3.2 [\[pairs.pair\]](#) after the definitions of swap:

```

template<class Self, class F>
    constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
        below);
55 Effects: equivalent to return std::apply(std::forward<F>(f), std::forward<Self>
    (self));

```

Add to 22.4.3 [\[tuple.tuple\]](#):

```

namespace std {
    template<class... Types>
    class tuple {
    public:
        // ...

        template<tuple-like UTuple>
            constexpr tuple& operator=(UTuple&&);
        template<tuple-like UTuple>
            constexpr const tuple& operator=(UTuple&&) const;

        // [tuple.swap], tuple swap
        constexpr void swap(tuple&) noexcept(see below);
        constexpr void swap(const tuple&) const noexcept(see below);

+   // [tuple.tuple.apply], calling a function with a tuple of arguments
+   template<class Self, class F>
+   constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
        below);
    };
}

```

And a new clause [\[tuple.tuple.apply\]](#) after 22.4.3.3 [\[tuple.swap\]](#):

```

template<class Self, class F>
    constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
        below);
1 Effects: equivalent to return std::apply(std::forward<F>(f), std::forward<Self>
    (self));

```

Add to 22.6.3.1 [\[variant.variant.general\]](#):

```

namespace std {
    template<class... Types>
    class variant {

```

```

public:
    // ...

    // [variant.status], value status
    constexpr bool valueless_by_exception() const noexcept;
    constexpr size_t index() const noexcept;

    // [variant.swap], swap
    constexpr void swap(variant&) noexcept(see below);

+   // [variant.visit], visitation
+   template<class Self, class Visitor>
+   constexpr see below visit(this Self&&, Visitor&&);
+   template<class R, class Self, class Visitor>
+   constexpr R visit(this Self&&, Visitor&&);
+   };
+   }

```

Add to 22.6.7 [\[variant.visit\]](#), after the definition of non-member visit:

```

template<class Self, class Visitor>
    constexpr see below visit(this Self&& self, Visitor&& vis);
template<class R, class Self, class Visitor>
    constexpr R visit(this Self&& self, Visitor&& vis);

```

- 9 *Effects:* Equivalent to `return std::visit(std::forward<Visitor>(vis), std::forward<Self>(self))` for the first form and `return std::visit<R>(std::forward<Visitor>(vis), std::forward<Self>(self))` for the second form.

Change the example in 22.14.6.4 [\[format.context\]](#)/8:

```

struct S { int value; };

template<> struct std::formatter<S> {
    size_t width_arg_id = 0;

    // Parses a width argument id in the format { digit }.
    constexpr auto parse(format_parse_context& ctx) {
        auto iter = ctx.begin();
        auto get_char = [&]() { return iter != ctx.end() ? *iter : 0; };
        if (get_char() != '{')
            return iter;
        ++iter;
        char c = get_char();
        if (!isdigit(c) || (++iter, get_char()) != '}')
            throw format_error("invalid format");
        width_arg_id = c - '0';
        ctx.check_arg_id(width_arg_id);
        return ++iter;
    }

    // Formats an S with width given by the argument width_arg_id.
    auto format(S s, format_context& ctx) {
-   int width = visit_format_arg([](auto value) -> int {
+   int width = ctx.arg(width_arg_id).visit([](auto value) -> int {

```

```

        if constexpr (!is_integral_v<decltype(value)>)
            throw format_error("width is not integral");
        else if (value < 0 || value > numeric_limits<int>::max())
            throw format_error("invalid width");
        else
            return value;
-    }, ctx.arg(width_arg_id));
+    });
    return format_to(ctx.out(), "{0:x<{1}}", s.value, width);
}
};

std::string s = std::format("{0:{1}}", S{42}, 10); // value of s is
            "xxxxxxxx42"

```

Add to 22.14.7.1 [\[format.arg\]](#):

```

namespace std {
    template<class Context>
    class basic_format_arg {
        // ...
    public:
        basic_format_arg() noexcept;

        explicit operator bool() const noexcept;

+    template<class Visitor>
+    decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);
    };
}

```

And:

```
explicit operator bool() const noexcept;
```

15 *Returns:* `!holds_alternative<monostate>(value)`.

```

template<class Visitor>
    decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);

```

16 *Effects:* Equivalent to: `return arg.value.visit(forward<Visitor>(vis));`

Add to 24.3.7.1 [\[array.overview\]](#):

```

namespace std {
    template<class T, size_t N>
    struct array {
        // ...

        // element access
        constexpr reference      operator[](size_type n);
        constexpr const_reference operator[](size_type n) const;
        constexpr reference      at(size_type n);
        constexpr const_reference at(size_type n) const;
        constexpr reference      front();
    };
}

```

```

constexpr const_reference front() const;
constexpr reference       back();
constexpr const_reference back() const;

constexpr T * data() noexcept;
constexpr const T * data() const noexcept;

+ // function application
+ template<class Self, class F>
+ constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
    below);
};
}

```

Add to 24.3.7.3 [\[array.members\]](#):

```
constexpr void swap(array& y) noexcept(is_nothrow_swappable_v<T>);
```

- 4 *Effects:* Equivalent to `swap_ranges(begin(), end(), y.begin())`.
- 5 [Note 1: Unlike the swap function for other containers, `array::swap` takes linear time, can exit via an exception, and does not cause iterators to become associated with the other container. — *end note*]

```

template<class Self, class F>
constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
    below);

```

- 6 *Effects:* equivalent to `return std::apply(std::forward<F>(f), std::forward<Self>(self))`;

Add to 26.5.4.1 [\[range.subrange.general\]](#):

```

namespace std::ranges {
    template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind
        K =
            sized_sentinel_for<S, I> ? subrange_kind::sized :
            subrange_kind::unsized>
        requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
        class subrange : public view_interface<subrange<I, S, K>> {
        public:
            // ...

            constexpr bool empty() const;
            constexpr make_unsigned_like_t<iter_difference_t<I>> size() const
                requires (K == subrange_kind::sized);

            [[nodiscard]] constexpr subrange next(iter_difference_t<I> n = 1) const
                &
                requires forward_iterator<I>;
            [[nodiscard]] constexpr subrange next(iter_difference_t<I> n = 1) &&;
            [[nodiscard]] constexpr subrange prev(iter_difference_t<I> n = 1) const
                requires bidirectional_iterator<I>;
            constexpr subrange& advance(iter_difference_t<I> n);

```

```
+ template<class Self, class F>
+     constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
        below);
    };
}
```

Add to 26.5.4.3 [\[range.subrange.access\]](#)

```
template<class Self, class F>
    constexpr decltype(auto) apply(this Self&& self, F&& f) noexcept(see
        below);
```

11 *Effects:* equivalent to `return std::apply(std::forward<F>(f), std::forward<Self>(self));`

§ 3 References

[P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this.
<https://wg21.link/p0847r7>

1. A single non-member function template is still superior to four member function overloads due to proper handling of certain edge cases. See the section on [SFINAE-friendly](#) for more information.↵